

Spot To Go: A Location-Based Android Application

for Restaurant Discovery with Video Integration

Final Project Report — Draft

Prepared for supervisor review and feedback

1. Introduction

This report documents the design, implementation, and current progress of “Spot To Go,” an Android application that allows users to discover nearby restaurants on an interactive map and access real-life video previews before deciding where to go. It follows on from the dissertation proposal submitted earlier in the project and reflects the system as it has actually been built, rather than only as it was originally planned.

The core problem motivating this project remains unchanged. Existing discovery platforms present restaurants through star ratings and short text reviews, both of which struggle to communicate atmosphere, food presentation, and overall experience. Short-form video, by contrast, has become one of the most trusted formats for conveying real-world experience (Dellarocas, 2003; Cheung et al., 2009). Spot To Go addresses this gap by pairing a live map of nearby restaurants with a direct link to a real video of each one, so a user can make a more informed decision before travelling anywhere.

This document is structured to give an honest account of progress: it restates the aim and objectives with their current status, revisits the literature review, describes the system as designed and implemented, evaluates what has been tested so far, updates the risk and ethical analysis in light of the real authentication system now in place, and closes with a clear statement of what remains to be built. Since the purpose of this draft is to gather feedback before the final phase of work, sections that describe incomplete features are marked explicitly rather than glossed over.

2. Aim and Objectives

Aim

The aim of the project is unchanged from the proposal: to design and develop an Android application that enables users to discover nearby restaurants through an interactive, location-aware map interface, and to access associated video content that supports informed decision-making.

Objectives and Current Status

The table below restates the seven objectives defined in the proposal and reports their current status, based on the implementation described in Sections 4 and 5.

No.	Objective	Status
1	Integrate the Google Maps SDK to render an interactive map centred on the user's real-time GPS location.	Achieved
2	Implement location-based restaurant discovery using the Google Places API.	Not yet started
3	Develop a restaurant detail screen presenting name, rating, distance, and description.	Achieved
4	Integrate video content from YouTube or TikTok per restaurant.	Achieved
5	Implement a navigation feature that launches Google Maps directions to a selected restaurant.	Achieved
6	Design a keyword-based search interface to filter nearby restaurant results.	Partially achieved
7	Evaluate the application through functional testing against each objective.	In progress

Objective 1 is complete: the map centres on the device’s real GPS position using the Fused Location Provider. Objectives 3, 4, and 5 are complete and have been exercised on a physical device. Objective 6 is partially achieved — a working search bar filters the restaurants currently loaded into the app, but it filters an in-memory list rather than passing the keyword to a live Places API query, since Objective 2 has not started yet. Objective 2 is the immediate next piece of work, discussed further in Section 8. Objective 7 is ongoing: functional checks have been carried out informally on a physical device during development, but the structured, objective-by-objective test documentation proposed in Section 5.7 of the original proposal has not yet been written up formally.

3. Critical Review of Related Literature

The development of Spot To Go continues to be informed by four interconnected areas of research: location-based services, restaurant recommendation systems, the role of video-based user-generated content in consumer decision-making, and Android application development. This section restates and, where relevant, updates the literature review from the proposal in light of the implementation decisions made since.

3.1 Location-Based Services

Location-based services (LBS) use the geographic position of a device to deliver contextually relevant information or functionality. Schiller and Voisard (2004) describe how GPS positioning, mobile networks, and application logic combine to enable this class of context-sensitive application. Dey (2001) extends this through context-aware computing — systems that adapt to the user’s physical environment automatically rather than through manual configuration. This principle is realised directly in the implemented app: the Fused Location Provider reads the user’s position automatically on the Map and Directions screens, with no manual location entry required anywhere in the interface.

Bao et al. (2012) shows that combining location data with user preference modelling improves the relevance of place-based recommendations. As in the proposal, this project does not implement preference modelling; the current architecture, in which the map consumes restaurant data

through a single repository interface, is designed so that such a model could be introduced later without changing the screens that depend on it.

3.2 Restaurant Recommendation Systems

Adomavicius and Tuzhilin (2005) identify content-based filtering, collaborative filtering, and hybrid approaches as the dominant paradigms for recommender systems. The proposal committed to a simplified content-based approach: keyword-based search against the Google Places API. At the time of writing, this specific integration has not yet been built — the search bar currently filters a small, hardcoded set of restaurants used as placeholder data during development of the surrounding screens. This distinction matters for how the literature review should be read: the architectural decision to filter by keyword is already implemented and working, and the remaining task is to point that same mechanism at live Places API data instead of the placeholder list. Once complete, the Places API’s built-in ratings and review counts will still provide the aggregated collaborative signal described in the proposal, without the application needing to implement that logic independently.

3.3 Video-Based User-Generated Content and Consumer Decisions

Dellarocas (2003) frames user-generated content as a digital extension of word-of-mouth communication with a measurable influence on purchasing decisions. Kaplan and Haenlein (2010) and Cheung et al. (2009) both identify video specifically as more credible and more effective than text at communicating atmosphere and experiential quality. This is the central premise the app now demonstrates in working form: from the restaurant detail screen, a user can open a real video of the restaurant with a single tap, and the app also offers a second path to a TikTok search for the same restaurant. Both actions hand off to the platform the user already trusts, rather than attempting to recreate video playback inside the app.

3.4 Android Mobile Application Development

Wasserman (2010) identifies device fragmentation, limited processing resources, and state management across screen transitions as recurring challenges in mobile development. The proposal

addressed state management through the Activity lifecycle; the implemented app instead addresses it through Jetpack Compose and Navigation Compose, in which each screen is a composable function managed by a single navigation host. This is a deviation from the proposal, and Section 5.6 explains the reasoning behind it in full. In short, the underlying challenge Wasserman describes — predictable state across navigation events — is still the driving design concern; only the specific mechanism used to solve it has changed.

The Google Play Services suite — the Maps SDK, the Fused Location Provider, and (prospectively) the Places API — remains the standardised, well-documented toolset underpinning the location-aware features of the app, exactly as anticipated in the proposal.

4. System Design

4.1 Application Architecture

The proposal specified a Java, single-Activity-per-screen architecture. The implemented application instead uses Kotlin with Jetpack Compose, in which every screen is a composable function rendered within one Activity (`MainActivity`) and connected through a single Navigation Compose host. This is a deliberate deviation from the original plan, made for three reasons. First, Kotlin is explicitly accepted as an alternative to Java in the project brief, so the change introduces no scope risk. Second, Compose is Google’s current recommended toolkit for new Android UI work, and using it demonstrates familiarity with present-day Android practice rather than a superseded one. Third, and most practically, Compose’s state-driven model turned out to simplify exactly the problem the proposal was most concerned about: passing the selected restaurant between the map, detail, video, and directions screens is handled by a single piece of shared state read by each screen, rather than by packaging data into each Activity’s launch `Intent` by hand.

Figure 1 shows the original hand-drawn prototype that the screen set was designed against. Eleven screens have been implemented in total: a Splash screen, Login and Registration, a Home screen, the Map, Restaurant Detail, a Video screen, a TikTok link screen, Directions, Contact Us, and a Privacy Policy screen. The last four of these — the TikTok link screen, Con-

tact Us, and Privacy Policy — extend beyond the six views sketched in the original prototype; they were added to give the video feature a second, working path (TikTok as well as YouTube) and to meet ordinary expectations of a publishable app (a contact channel and a privacy statement), without changing the core flow the prototype describes.

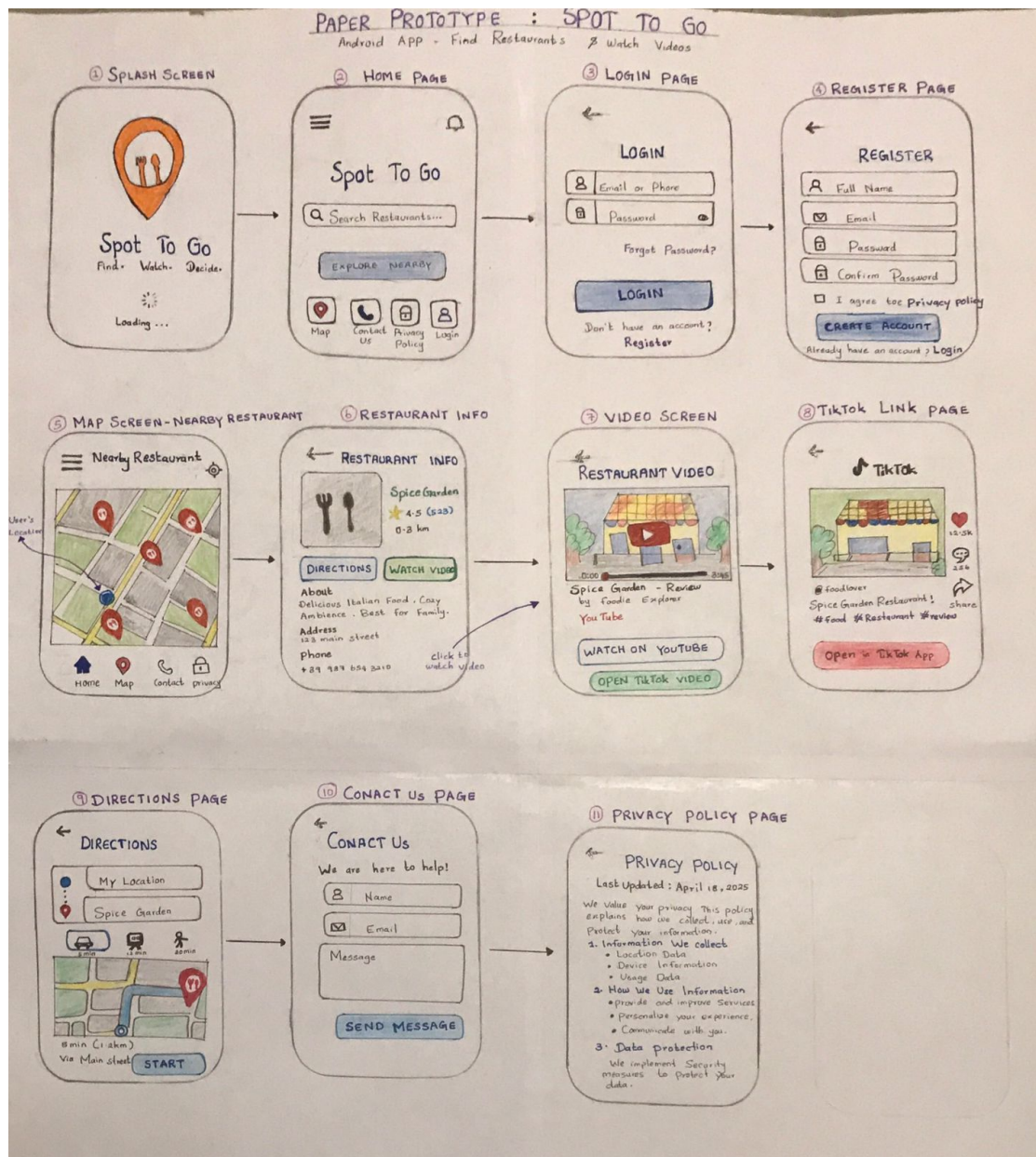


Figure 1: Original hand-drawn prototype of the Spot To Go application, showing the planned flow across login, home search, the map with restaurant markers, restaurant detail, video playback, and directions. This layout directly informed the screen set described above.

Figure 2 traces the same eleven screens as an explicit navigation map, showing every transition a user can actually make, including the two secondary screens (Contact Us and Privacy Policy) and the bottom navigation bar added during testing (Section 6.1) that lets a user return from the

Map screen to Home.

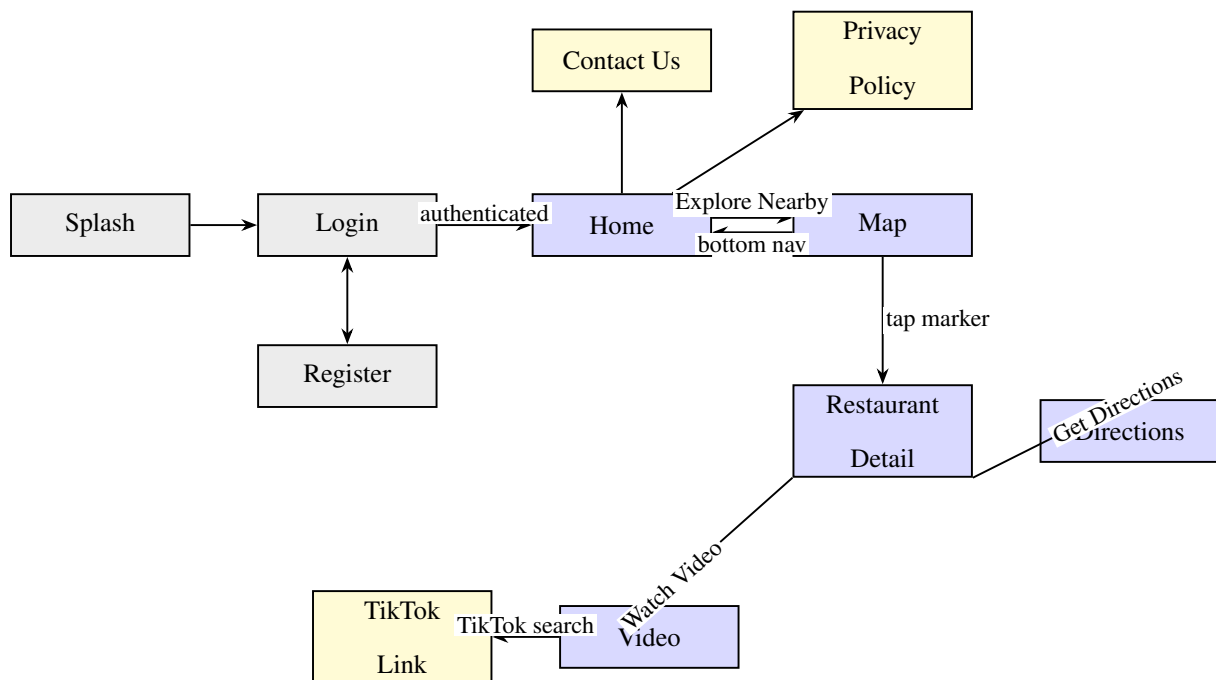


Figure 2: Screen navigation map for Spot To Go. Gray nodes are the authentication screens, blue nodes are the core discovery flow, and yellow nodes are secondary screens reached from Home. The Map-to-Home bottom navigation link was added in response to a usability issue found during testing (Section 6.1).

4.2 Software Architecture

Figure 3 shows the application in four layers rather than as a sequence of screens. The point of drawing it this way is to make explicit a design decision referenced throughout this report: every screen depends only on a repository interface, never on where that repository's data actually comes from. This is why Section 8's highest-priority task — replacing the in-memory restaurant list with a live Google Places API client — is described as requiring no change to the Map screen itself; the diagram shows that dependency boundary directly.

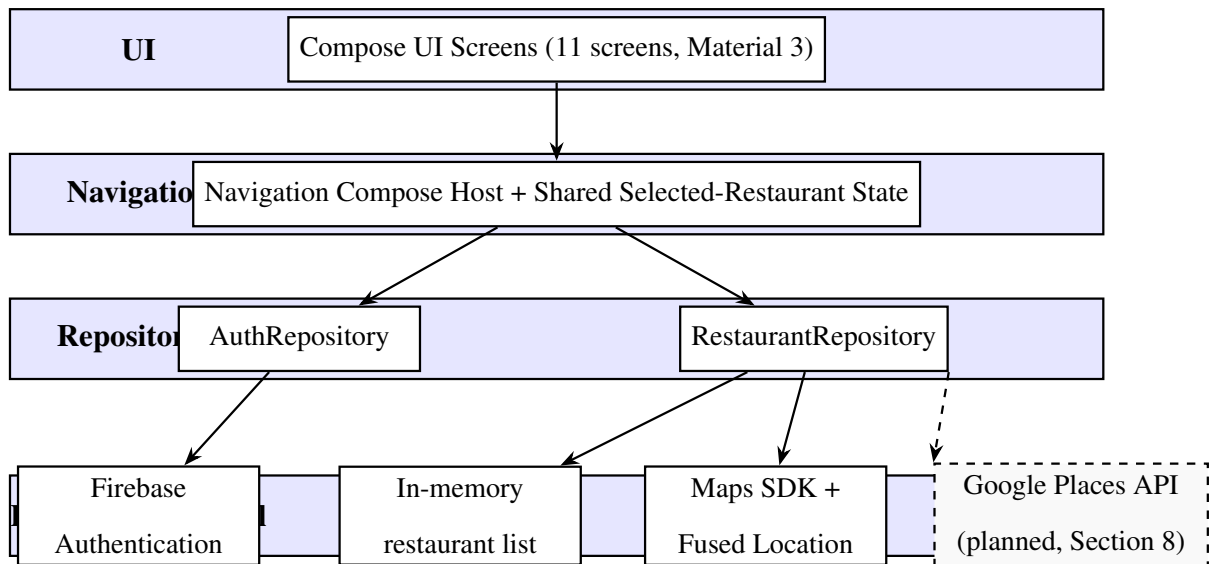


Figure 3: Layered software architecture of Spot To Go. Screens depend only on the repository interfaces in the layer below; `RestaurantRepository` currently returns an in-memory list, and the dashed box shows where the planned Google Places API client will attach without requiring any change above the repository layer.

4.3 Data Flow

Figure 4 shows the data flow through the application as it is currently implemented. A user first authenticates through Firebase; once signed in, the Home screen leads to the Map, which currently reads from an in-memory restaurant list rather than a live query — this stand-in is discussed further in Section 5.3 and Section 8. Selecting a marker opens the Restaurant Detail screen, from which the user can either watch a video or request directions; both actions hand off to an external app through an `Android Intent` rather than being handled inside Spot To Go itself.

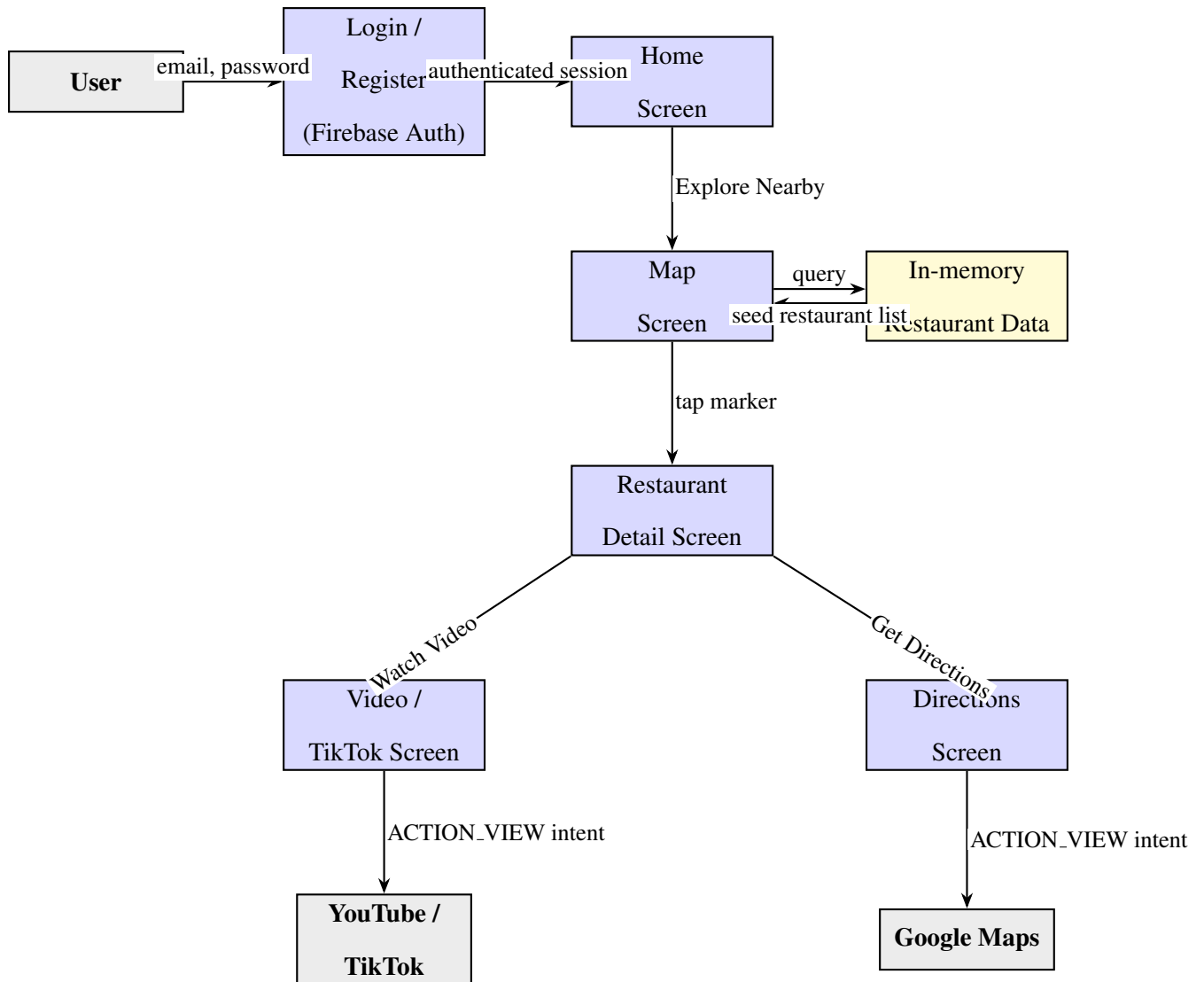


Figure 4: Current data flow through Spot To Go. The in-memory restaurant store is a temporary stand-in for the planned Google Places API integration described in Section 8.

5. Implementation

5.1 Screens

Figure 5 shows all eleven implemented screens. Every screen is built as a Jetpack Compose function using Material 3 components, styled with a single Deep Orange (#FF5722) theme applied consistently across the app.

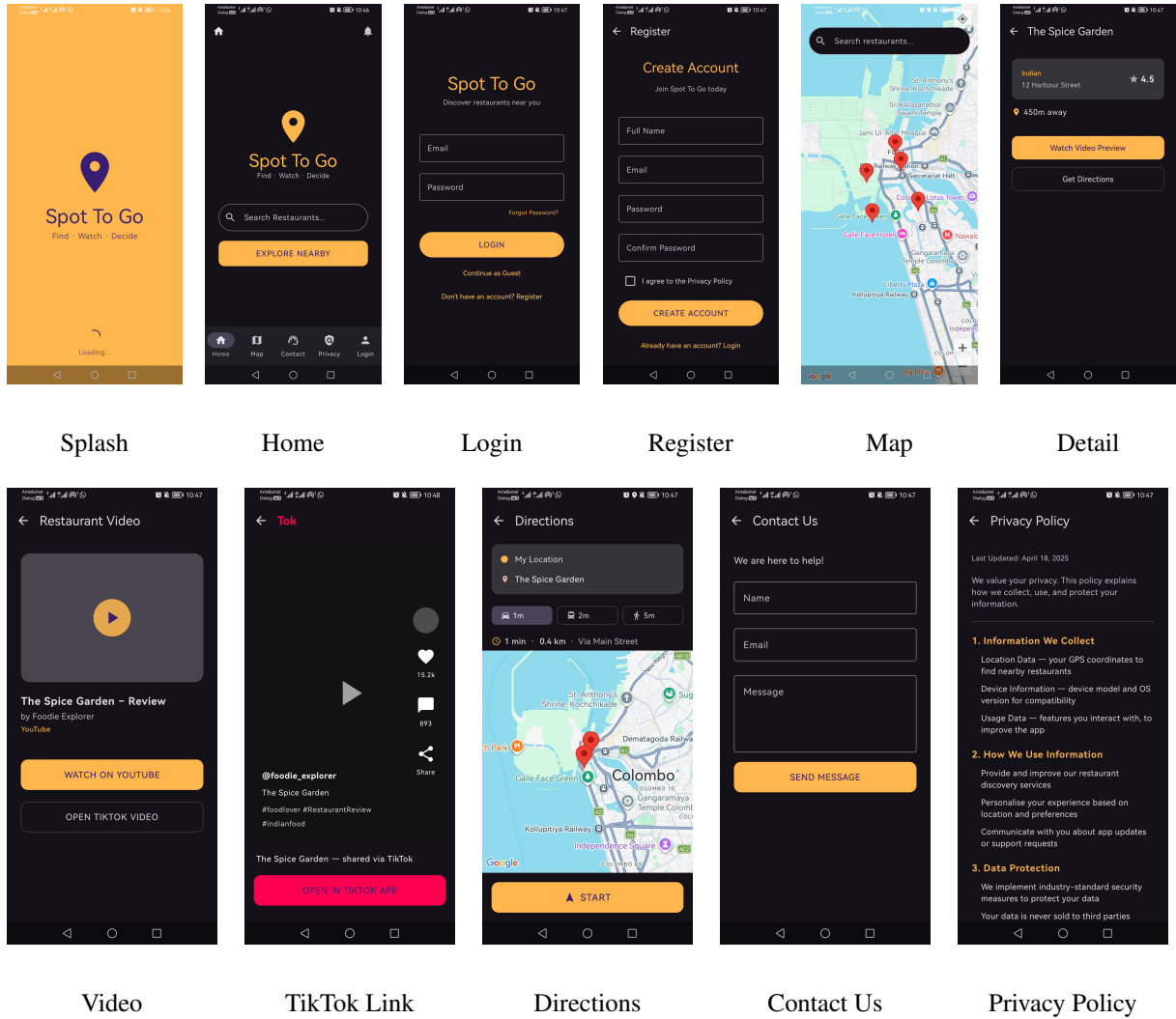


Figure 5: All eleven implemented screens, matching the flow shown in Figure 1.

5.2 Location and Mapping

The Map screen uses the Fused Location Provider (`FusedLocationProviderClient`) to read the device's last known location, after requesting the `ACCESS_FINE_LOCATION` runtime permission through the Accompanist Permissions library. The map itself is rendered with the Maps Compose library (`com.google.maps.android.compose`), which wraps the native Google Maps SDK for use inside a composable. Each restaurant is shown as a marker with the restaurant's name and a snippet combining its cuisine and rating; tapping a marker opens the Restaurant Detail screen for that restaurant. The Directions screen uses the same pattern independently, so that if the user's location changes between viewing the map and requesting directions, the directions screen still reflects a current position.

5.3 Restaurant Data

Restaurant data is currently served by `RestaurantRepository`, which returns a fixed, in-memory list of five restaurants (spanning Indian, Chinese, Italian, American, and Japanese cuisine) positioned at small fixed offsets from the user’s real coordinates, so that markers appear plausibly “nearby” regardless of where the app is run. Each restaurant record carries a name, cuisine, rating, computed distance, address, coordinates, and a real YouTube video URL. This repository was deliberately built behind a single interface consumed by the Map screen, so that replacing it with a live Google Places API client — the next piece of work, described in Section 8 — requires no change to the Map screen itself.

5.4 Video and Directions

Rather than embedding video playback inside the app with a `WebView`, the Video screen dispatches the restaurant’s YouTube URL through an Android `Intent` with action `ACTION_VIEW`, which the operating system hands to the installed YouTube app or, failing that, a browser. A second button on the same screen leads to a TikTok Link screen, styled to resemble a TikTok post, which in turn opens a TikTok search for the restaurant’s name through the same `ACTION_VIEW` mechanism. Directions follow an equivalent pattern: the Directions screen builds a Google Maps web URL from the user’s current coordinates and the restaurant’s coordinates, with the selected transport mode (car, bus, or walk) encoded in the URL, and dispatches it through `ACTION_VIEW` so that the installed Google Maps app opens with turn-by-turn directions already configured.

5.5 Authentication

Login and registration are backed by Firebase Authentication through a single `AuthRepository` object, exposing `register`, `login`, `logout`, and an `isLoggedIn` check. Registration creates a Firebase user with email and password and sets the user’s display name; login signs an existing user in through the same mechanism. The Map screen checks `isLoggedIn` itself and redirects to the Login screen if no session is active, so that the map — and everything reachable from it — is gated behind a real sign-in rather than a hardcoded credential check as originally

proposed. This has been tested end-to-end on a physical Android device: registering a new account, being signed in automatically, and reaching the map.

Figure 6 sets out that tested flow as a sequence diagram, showing which calls are handled inside the app and which are delegated to Firebase.

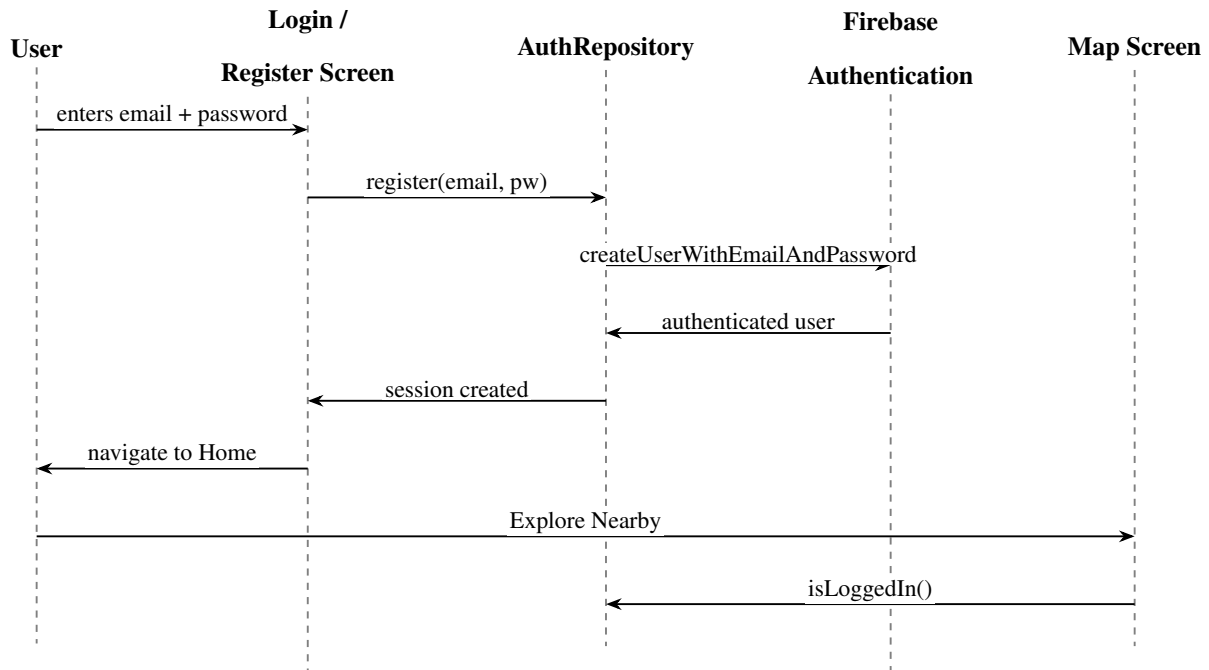


Figure 6: Sequence diagram of the tested authentication flow, from account registration through to the Map screen’s own session check. AuthRepository is the only component that talks to Firebase directly; the Map screen re-checks isLoggedIn independently so it can never be reached without a valid session, regardless of how it was navigated to.

5.6 Navigation and State

All eleven screens are connected through a single Navigation Compose host defined in MainActivity, with the splash screen as the start destination. Rather than passing the selected restaurant as a navigation argument on every route, the app holds it as a single piece of shared Compose state at the navigation level, set whenever a restaurant is chosen from the map. The Detail, Video, TikTok, and Directions screens all read from this same value, so the currently selected restaurant survives navigating between them.

5.7 Deviations from the Original Proposal

Four implementation choices depart from the proposal, summarised here for transparency:

- **Kotlin and Jetpack Compose instead of Java and Activities.** Justified above in Section 4.1; permitted by the project brief and better suited to sharing state between screens.
- **Real Firebase Authentication instead of hardcoded credentials.** The proposal treated login as a demonstration screen with no live backend, to avoid handling real personal data. The implemented app uses real Firebase Authentication instead, because gating the map behind a genuine sign-in makes the “informed decision-making” narrative of the app more credible during a live demonstration, and because Firebase Authentication is a managed service that stores credentials securely without the project needing to run its own user database. Section 6.2 revisits the ethical implications of this change.
- **Intent-based video playback instead of an in-app WebView.** Handing playback to the installed YouTube or TikTok app is simpler to implement, matches how users already expect to watch video content on their phone, and avoids the extra permissions and edge cases that come with embedding a web view.
- **A Google Maps web-intent URL instead of a `geo:` URI for directions.** The web-intent form still opens the native Google Maps app when it is installed, but falls back gracefully to a browser when it is not, which is a safer default for a device configuration that has not been tested in advance.

6. Testing and Evaluation

6.1 Functional Testing

Testing to date has been carried out informally on a physical Android device rather than through the structured, objective-by-objective test plan proposed earlier, and this remains an open task noted in Section 8. That said, the core flow — opening the app, registering an account, being signed in automatically, viewing the map, opening a restaurant’s detail screen, watching its video, and requesting directions — has been exercised directly and works as intended.

This testing surfaced five issues, all of which have since been fixed:

- The on-screen keyboard was covering the password field on the Login and Registration screens; resolved by applying `imePadding` so the form scrolls clear of the keyboard.
- There was no way to check a password had been typed correctly before submitting; resolved by adding a show/hide toggle to both password fields.
- The Map screen had no way back to the rest of the app other than the phone’s back button, which simply exited the app; resolved by adding a bottom navigation bar consistent with the Home screen.
- The Home screen’s navigation bar showed a “Login” option even when a user was already signed in; resolved by having both navigation bars read the same `isLoggedIn` state.
- The Login screen’s “Continue as Guest” option bypassed Firebase Auth entirely and, once the Map screen was gated on a signed-in session, sent guest users into a redirect loop; the button was removed. The Login screenshot in Figure 5 was captured before this fix and still shows the button, which has since been removed from the app.

6.2 Usability Considerations

A lightweight heuristic pass, following Nielsen (1994), was applied informally during the fixes above rather than as a separate, scheduled review. Of the ten heuristics, four are most directly relevant to this app: visibility of system status, match between the system and the real world, user control and freedom, and error prevention. The user-control issue identified above (the map’s earlier dead end) has been resolved; a related, smaller issue — that pressing back on the map screen still exits the app immediately, since it is the first screen reached after login — remains open and is listed as a task in Section 8. The formal second stage proposed earlier, in which three to five participants complete a defined task list and a System Usability Scale questionnaire, has not yet been run and is scheduled for after the Places API integration is complete, so that participants test against live restaurant data rather than the placeholder list.

7. Analysis of Risks and Ethical Issues

7.1 Technical Risks

The Google Places API request quota risk described in the proposal has not yet materialised, since the app currently runs entirely on in-memory data and makes no calls to the Places API. This risk becomes active as soon as the integration described in Section 8 begins, and will be managed as originally planned: by caching responses where they are stable and by controlling request frequency during testing.

The risk that linked video content may be removed or become unavailable still applies to the five video URLs currently seeded into the app, and is mitigated in the same way as proposed — by choosing stable, established videos for demonstration purposes.

Device fragmentation remains a live consideration. The app currently targets a minimum SDK of API level 24 and a target SDK of API level 36, which covers the large majority of active Android devices; testing so far has been carried out on a single physical device, and testing across a wider range of screen sizes and OS versions remains outstanding.

7.2 Ethical Considerations

This section is the one most significantly affected by the deviation described in Section 5.7. The proposal stated that login would be demonstration-only, collecting no real personal data. The implemented app instead uses real Firebase Authentication, which does collect a genuine email address and password for any user who registers. This changes the ethical position of the project and needs to be stated plainly rather than carried over unchanged from the proposal.

In practice, the risk this introduces is small and well-managed. Firebase Authentication is a managed identity service operated by Google; passwords are never stored or handled directly by the application's own code, and no additional personal data — beyond the email address and display name required to sign in — is collected or stored anywhere in the app. No Firestore database or other persistent store of user data is used. Location data continues to be used only transiently, to centre the map and compute the placeholder restaurant positions, and is not stored

beyond the current session. Because the accounts created during testing are the developer's own test accounts rather than data from human participants, and because no analytics or third-party tracking is integrated, the change does not introduce a need for formal ethical approval; it does, however, mean that any demonstration or further testing involving other people's real credentials should be limited to consenting testers who understand that a genuine account is being created.

8. Current Progress and Remaining Work

8.1 Summary of Progress

All eleven planned screens are implemented and connected through a single navigation flow. The map renders live using the device's real GPS location and the Google Maps SDK. Firebase Authentication is live and has been tested end-to-end on a physical device, including the UX fixes listed in Section 6.1. Video and directions both work by handing off to external apps through Android intents, and the search bar filters the currently loaded restaurant list in real time.

8.2 Remaining Work

Three pieces of work are outstanding, in priority order:

1. **Google Places Nearby Search API.** Replace `RestaurantRepository`'s in-memory list with a new repository backed by a live Nearby Search call (type `restaurant`, radius 1500 metres), returning real restaurants around the user's location. Because the Map screen already depends only on the repository's interface, this screen requires no changes of its own.
2. **Live search.** Once real restaurant data is flowing, connect the existing search bar to the Places API's keyword parameter with a short debounce, replacing the current client-side filter over the placeholder list, and show a loading indicator while a search request is in flight.
3. **Back-press polish and formal testing.** Add a press-back-again-to-exit confirmation on the Map screen, and carry out the structured functional test plan and the participant-based

usability testing (Section 6.2) that were deferred until real restaurant data is available to test against.

Figure 7 sets out the same three items as a dependency chain rather than a dated schedule, since no fixed dates have been committed to at this stage of the project. The ordering reflects a genuine dependency, not just a preference: live search cannot be connected to the Places API's keyword parameter (item 2) until that API call exists at all (item 1), whereas the back-press and testing work (item 3) is independent and could, in principle, proceed in parallel.

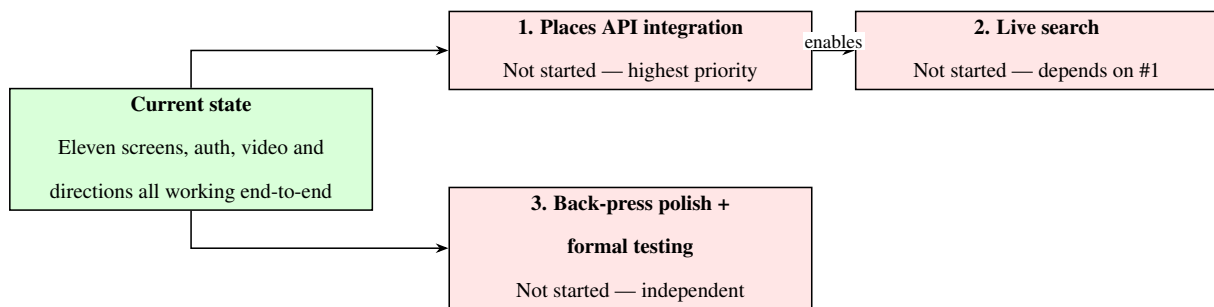


Figure 7: Remaining work as a dependency chain rather than a dated roadmap. The Places API integration (Section 8, item 1) blocks live search (item 2); formal back-press and usability testing (item 3) does not depend on either and could run in parallel.

9. Conclusion

Spot To Go has progressed from a proposal into a working Android application. Every screen from the original prototype has been built, styled consistently, and connected into a single, tested navigation flow; a real authentication system gates access to the map; and the map itself renders the user's true location alongside restaurants that can be explored through a real video preview or turned into real driving, walking, or transit directions with a single tap. The one substantive gap against the original objectives is the live Google Places API integration, which remains the clearly scoped next step — the surrounding architecture was built specifically so that this change would not require reworking any of the screens already in place.

The purpose of this draft is to put that progress in front of a reviewer before the remaining work

begins, so that feedback on the screens, the architecture, and the ethical position described in Section 6.2 can shape how the final phase of the project is prioritised.

References

- Adomavicius, G. and Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749.
- Bao, J., Zheng, Y., and Mokbel, M. F. (2012). Location-based and preference-aware recommendation using sparse geo-social networking data. In *Proceedings of the 20th International Conference on Advances in Geographic Information Systems*, pages 199–208. ACM.
- Cheung, C. M. K., Lee, M. K. O., and Rabjohn, N. (2009). The impact of electronic word-of-mouth: The adoption of online opinions in online customer communities. *Internet Research*, 18(3):229–247.
- Dellarocas, C. (2003). The digitization of word of mouth: Promise and challenges of online feedback mechanisms. *Management Science*, 49(10):1407–1424.
- Dey, A. K. (2001). Understanding and using context. *Personal and Ubiquitous Computing*, 5(1):4–7.
- Kaplan, A. M. and Haenlein, M. (2010). Users of the world, unite! the challenges and opportunities of social media. *Business Horizons*, 53(1):59–68.
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann, San Francisco, CA.
- Schiller, J. and Voisard, A. (2004). *Location-Based Services*. Morgan Kaufmann, San Francisco, CA.
- Wasserman, A. I. (2010). Software engineering issues for mobile application development. In *Proceedings of the FSE/SDP Workshop on the Future of Software Engineering Research*, pages 397–400. ACM.